UMBRAL VR: informe de avance

Sala de ventas inmersiva en realidad virtual para departamentos en preventa (Meta Quest 3/3S).

Qué se logró

Servidor (`server/`)

- API en NestJS 11 con dos endpoints: `GET /health` y `GET /catalog`.
- El catálogo se sirve desde `server/seed/catalog.json` (unidades, acabados y precios en bolivianos).
- Compila sin errores y pasa sus 4 pruebas automáticas (precio base positivo, acabado por defecto sin diferencia, ids únicos, fecha de vigencia).

Cliente (`client/`)

- Proyecto Unity 6 (6000.6.8f1) con URP, OpenXR y XR Interaction Toolkit.
- Lógica del negocio aislada en `Umbral.Core` (C# puro, sin dependencias de Unity):
  - cálculo de precio = base de la unidad + diferencias de acabados, en bolivianos enteros;
  - consultas sobre el catálogo.
- Capa de red y caché (`Umbral.Net`): el catálogo carga desde el servidor, luego caché en disco y, como último recurso, un catálogo incluido en la app. La app funciona con el servidor apagado.
- 10 pruebas automáticas en Unity, todas aprobadas, sin errores de compilación.

Ingeniería

- Repositorio Git con ramas cortas y `main` siempre desplegable.
- Flujos de integración continua para servidor y cliente (GitHub Actions).
- Documentación: diseño del juego (GDD), plan por semanas y decisiones de arquitectura.

Estado de verificación

Componente	Resultado
Servidor: instalación y compilación	OK
Servidor: pruebas	4 de 4
Cliente: resolución de paquetes XR	OK
Cliente: pruebas EditMode	10 de 10

Siguiente

1. Escena de arranque con el precio de la unidad A-301 en la muñeca izquierda.
2. Configuración Android para Quest y primera medición de rendimiento.
3. Build instalable en el visor y despliegue del servidor de pruebas.